# Core Concepts

> Part of the React curriculum. This document covers components, props, state, hooks, and the custom-hook pattern for data fetching.

## 1. What a single-page application (SPA) is, and why use one

A single-page application loads once, then updates the page in-place (via JavaScript) as the user navigates and interacts, rather than requesting a whole new HTML page from a server for every action. **React** is a library for building the UI of an SPA out of reusable **components** — small, self-contained pieces of UI, each responsible for rendering some piece of the page from data passed into it.

Choosing this approach over a single-process dashboard framework (Streamlit curriculum) trades faster initial development for a richer, more customizable UI, real component reuse, and a UI layer that's fully decoupled from — and testable independently of — the backend serving its data.

## 2. Components and props

```
interface KpiCardProps {
  label: string;
  value: string;
}

function KpiCard({ label, value }: KpiCardProps) {
  return (
    <div className="kpi-card">
      <span className="kpi-label">{label}</span>
      <span className="kpi-value">{value}</span>
    </div>
  );
}

// used as: <KpiCard label="Total" value="1,234" />
```

A component is just a function that takes **props** (its inputs, passed like HTML attributes) and returns what should be rendered. Components compose — a page is usually a component that renders several smaller components, each with its own props. Props are **read-only** from the receiving component's perspective — a component should never mutate its own props directly; if it needs to change something, that change belongs in the parent that owns the data (or in the component's own local state, §3).

# 3. State

**State** is data a component owns and can change over time, triggering a re-render when it does:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);   // [currentValue, setterFunction]

  return <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>;
}
```

`useState(initialValue)` returns a pair: the current value, and a setter function. Calling the setter schedules a re-render with the new value — React does **not** mutate the existing value in place; it replaces it, which is why you call `setCount(count + 1)` rather than `count++`.

## 3.1 Controlled vs. uncontrolled inputs

A **controlled** input's value is driven entirely by React state — the input always shows exactly what state says it should, and every keystroke updates that state:

```
function ControlledInput() {
  const [value, setValue] = useState('');
  return <input value={value} onChange={(e) => setValue(e.target.value)} />;
}
```

An **uncontrolled** input instead manages its own internal DOM state, and you read its current value only when you need it (typically via a `ref`, §6):

```
function UncontrolledInput() {
  const inputRef = useRef<HTMLInputElement>(null);
  const handleSubmit = () => console.log(inputRef.current?.value);
  return <input ref={inputRef} defaultValue="" />;
}
```

Controlled inputs are the more common default in React — they keep a single source of truth (React state) rather than letting the DOM and your application state potentially disagree — but uncontrolled inputs are simpler for cases where you genuinely don't need to react to every keystroke (e.g., a form you only read on submit).

# 4. Effects

```
import { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    // runs after render, and again whenever a dependency in the array below changes
    document.title = `Count: ${count}`;
  }, [count]);

  return <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>;
}
```

`useEffect` lets a component "hook into" the browser/DOM or perform side effects (fetching data, setting a document property, subscribing to an external event) that aren't a pure function of props/state alone. Its dependency array (`[count]` above) controls when the effect re-runs; an empty array (`[]`) means "run once, after the first render only"; omitting the array entirely means "run after every single render" (rarely what you want).

### 4.1 Cleanup functions

An effect can return a **cleanup function**, which React runs before the effect re-runs (due to a dependency changing) and when the component unmounts:

```
useEffect(() => {
  const timer = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(timer);   // cleanup: stop the timer when this effect is superseded or the component unmounts
}, []);
```

Forgetting cleanup for anything that persists beyond a single render (timers, subscriptions, in-flight requests) is one of the most common sources of subtle bugs and memory leaks in React apps.

## 5. A custom hook for data fetching

The `useState` / `useEffect` pattern above is exactly what's needed to fetch data from an API and track its loading/error/success state — and since almost every page in a data dashboard needs this same pattern, it's usually extracted into a reusable **custom hook**:

```
import { useEffect, useState } from 'react';

interface AsyncState<T> {
  data: T | null;
  error: string | null;
  loading: boolean;
}

function useAsync<T>(fn: () => Promise<T>, deps: unknown[]): AsyncState<T> {
  const [state, setState] = useState<AsyncState<T>>({ data: null, error: null, loading: true });

  useEffect(() => {
    let cancelled = false;
    setState((s) => ({ ...s, loading: true, error: null }));
    fn()
      .then((data) => { if (!cancelled) setState({ data, error: null, loading: false }); })
      .catch((err) => { if (!cancelled) setState({ data: null, error: err.message, loading: false
}); });
    return () => { cancelled = true; };   // cleanup: runs when deps change or the component
unmounts
  }, deps);

  return state;
}

// used in a page component:
function OverviewPage() {
  const { data, error, loading } = useAsync(() => fetchOverview(), []);
  if (loading) return <Spinner />;
  if (error) return <ErrorMessage text={error} />;
  return <div>{/* render data */}</div>;
}
```

The `cancelled` flag in the cleanup function matters: without it, a slow request that resolves *after* the component has already unmounted (or after a newer request has started, because `deps` changed) would still call `setState` on a component that's no longer mounted — a common source of warnings and subtle bugs in real React apps. This loading/error/data three-state pattern is worth internalizing — it's the shape almost every data-driven page in an app like this will need. A custom hook is just a regular function whose name conventionally starts with `use` and which itself calls other hooks — this is what makes it reusable across components rather than copy-pasted logic.

## 6. Other hooks worth knowing

```
import { useContext, useMemo, useCallback, useRef } from 'react';
```

- **`useRef`** — holds a mutable value that persists across renders *without* triggering a re-render when it changes (unlike state). Commonly used to reference a DOM element directly (§3.1's uncontrolled input example), or to store a value you need to remember between renders but that shouldn't itself cause a re-render (like the `cancelled` flag's conceptual cousin, if you needed it accessible outside the effect closure).
- **`useMemo`** — recomputes an expensive derived value only when its dependencies change, rather than on every render: `const sorted = useMemo(() => [...items].sort(), [items])`. A performance optimization — reach for it once you've identified an actual expensive recomputation happening unnecessarily often, not as a default on every derived value.
- **`useCallback`** — like `useMemo`, but memoizes a *function* rather than a value; useful when passing a callback down to a child component that itself uses `React.memo` or a dependency array that would otherwise see a "new" function reference on every render.
- **`useContext`** — reads a value provided higher up the component tree via `React.createContext` — see Routing & API Integration, §5 for how this is used for state that needs to be shared broadly without manually passing props through every intermediate component ("prop drilling").

# 7. Design systems: tokens, themes, and components

A **design system** is more than a component library: it is the set of *decisions* about how an interface looks and behaves (colours, spacing, type, component states, accessibility rules) captured as data and code, so every application that consumes it looks and behaves consistently. A typical one has four parts: **design tokens** (the named values), a **component library** (buttons, tables, charts, app shell), **themes** (alternative token sets), and **conventions plus tests** that keep all of it coherent. This project's front end is built on one, the Syena Design System (`@syena/ds`, the `syena-design-system` repository), a separately maintained repository shared by several apps; its own reference is `FOUNDATIONS.md` in that repository (design principles, the generated scales, the colour system, the app shell, and a specification for each component, chart and table). This section explains the principles; consult that document for specifics.

## 7.1 Tokens are the contract

Rather than writing `color: #0A6E8C` in an app, a design system exposes **semantic tokens** as CSS custom properties, here all under the `--syn-` prefix (`--syn-color-*` for text and surface roles, `--syn-data-panel-*` for chart panels). The pipeline runs in one direction:

| Stage | What it holds |
|---|---|
| Primitives | Scales and colour ramps *generated from formulas* (`FOUNDATIONS.md` §2–3), kept as JSON; never emitted as CSS variables |
| Semantic tokens | Role-named values (`vocabulary.json` lists the only words a token name may use), resolved per theme and emitted as `--syn-*` custom properties |
| Component CSS | A co-located CSS Module per component that consumes semantic tokens only; no colour literals |
| Themes | One token set per theme, scoped under `[data-syn-theme="…"]`, overriding semantic tokens only |

The generated token files are **not committed**: a build step (`npm run tokens`) produces `tokens.css` and `tokens.ts` from the sources, so a checkout must run it before anything can import the stylesheet (Deployment, §8). The consequences for an application:

- **Use semantic tokens, never raw colours,** in your own CSS. A hard-coded hex works until the user switches theme, and then it is the one thing on the page that doesn't change.
- **Add app-level tokens sparingly and by purpose** (a warning colour, a link colour that must differ per theme), named for what they mean, defined once per theme. This app adds only two.
- **Prefer a semantic name to a visual one,** so that "weak text" can be retuned per theme to meet contrast without touching any component.

## 7.2 Themes: one switch, applied once

A theme is a set of token values scoped to an attribute on an element:

```
<div data-syn-theme="tidewater"> …the whole app… </div>
```

Two themes ship: `night` (dark, the default analytical theme) and `tidewater` (light). Apply a theme **once, at the root of the app**; individual pages never opt in or out. In React that is a single provider, `SynThemeProvider`, which sets the attribute (and, with `syncDocument`, mirrors it on the document element so the page background and portals such as dropdowns follow the theme). Switching between dark and light is then changing one value (see Patterns for Data-Heavy Dashboards, §7). Principles that came out of building and adopting them:

- **Polarity, not per-theme hand tuning.** A theme is defined by a few seed colours plus a polarity (dark or light); "toward emphasis" and "elevation" are derived from the polarity, so a third theme is added by supplying seeds rather than re-choosing every value.
- **Choose brand hues that don't collide with semantic colours.** If green and red already mean "fell" and "rose" in the product, a green brand competes with that meaning; a water/atmosphere hue avoids it and keeps magnitude colour ramps unambiguous. The design system goes further and *reserves* climate semantics for the data.
- **Chart panels stay dark in both themes.** A vivid categorical palette is validated on one kind of ground, so chart components read `--syn-data-panel-*` tokens rather than the page background. Put your own chart-adjacent elements on those tokens inside a chart panel.
- **Contrast is validated per theme.** Text and surface tokens are tuned for each theme's own surfaces, and the token build emits a contrast matrix, so a theme can't ship a pairing that fails.
- **A theme must set its own text colour at its root,** or a component without an explicit `color` inherits a value from the other theme. If a component is unreadable in only one theme, check inheritance before assuming the theme is incomplete.

## 7.3 Namespacing and required props

Everything the design system emits is namespaced: tokens under `--syn-`, theme and state selectors under `data-syn-*` attributes, component classes as CSS Modules (locally scoped, so an app's `.button` can't collide with the library's). A `check:allowlist` script in the design system's repository enforces the prefix, the token vocabulary, class names and the ban on colour literals. Components that need per-app branding take it as **required props**

**with no default** (a footer's `copyright`, for example). A forgotten value is then a *type error*, not a silent brand leak. The lesson for any shared library: make the thing you want consumers to supply impossible to omit.

## 7.4 Consuming a design system

How the application takes the dependency matters operationally:

- **Sourced vs. published.** Here the app aliases `@syena/ds` straight to a sibling checkout's `src/index.ts` (and `@syena/ds/styles.css` to `src/styles.css`) — no published package, no version to bump. The checkout is **pinned** to a commit recorded in the app's repository (`syena-design-system.pin`), and a setup script clones it at that commit and generates its tokens, so the build is reproducible even though the dependency is not versioned. Moving the pin is a deliberate, reviewable change; the checkout must be at the pin (and its tokens generated) *before* the app is built (Deployment, §8). A published, versioned package is the opposite trade-off: slower to adopt, but needs no pin file or setup script.
- **Aliases live in three places.** The bundler config, the test-runner config and the TypeScript configs each carry the alias; an environment variable (`SYN_DS_PATH`) relocates the first two, but the type check reads fixed relative paths, so the sibling position still matters.
- **One React instance.** A component library running inside your bundle must share your copy of `react` / `react-dom`; two copies break hooks. Alias both explicitly.
- **Import the stylesheet once, at the entry point.** `@syena/ds/styles.css` carries the tokens and the base layer (reset, focus, typography, utilities); component styles arrive with the components.
- **Fix gaps upstream, not per app.** When a component is unreadable in a theme, an icon has no contrast, or a header wraps on a phone, the fix belongs in the design system (and its tests) so every consumer benefits; the app records the request. App-specific decisions belong in the app's own docs, not the shared ones.

## 7.5 Accessibility and testing are part of the system

A design system earns trust by testing what it ships:

- **Stories are tests.** Each component has stories, and each story is an automated test that mounts it, runs any `play` interaction script (keyboard navigation, focus traps, clamping, open/close), and runs an automated accessibility check (axe) configured to *fail* on a violation. Interaction logic is tested in the story rather than in a separate file.
- **Contrast is a token property.** Fixing a shared token fixes every component that uses it; compute replacements with the WCAG relative-luminance formula, and let the generated contrast matrix catch regressions.
- **Keyboard behaviour is designed.** For example, a data grid suppresses cell focus (tab-stopping every cell of an informational table is worse for keyboard users) and offers an explicit row-activate handler that wires click *and* Enter/Space to the same action — use it rather than a mouse-only row-click handler.
- **In the app, stub the heavy parts.** The chart component wraps a browser-only plotting library; page tests stub it and leave its own rendering to the design system's tests (Testing, §6).

## 7.6 Gotchas worth knowing before you theme anything

These cost real debugging time and generalise beyond this library:

- **Custom-property cascade is per element.** A token you set at the theme root is silently ignored if a stylesheet re-declares the same property on a *nearer* element (a navigation container, say). Target the same or a more specific selector as the rule you are overriding.
- **Some colours are written as inline SVG attributes** by a charting library and can't be reached by a normal CSS rule; they need a `fill: … !important` override.
- **Fonts need every font-family token, plus the grid library separately.** Repointing only the primary family leaves other text styles — and AG Grid, which ignores them all — in the old font; the consumer must also import the font files, and a different typeface may need a `font-size-adjust` to match optical size.
- **Fixed-height headers and long wordmarks.** Truncate with `max-width` plus ellipsis, not `min-width: 0`: overriding a grid item's automatic minimum can collapse its whole column.
- **Header-adjacent controls must not inherit the header's ink,** or a dark header can swallow a button whose colour resolves to the same value.

## 8. Core libraries summary

| Library | Used for |
|---|---|
| **React** | components, hooks, state, rendering |
| **TypeScript** | static typing across components and their props |

## See also

- React curriculum index
- Routing & API Integration — building on these concepts to navigate between pages and talk to a backend